

Completing MiniVector: Testing, Debugging & Memory Errors

CSCI 235 — Class #6

Tong Yi

Hunter College, CUNY

Thursday, September 17, 2026

Where We Are

- MiniVector **grows** correctly, with amortized $O(1)$ `push_back` (Day 4).
- MiniVector **copies** correctly: the Rule of Three — destructor, copy constructor, `operator=` (Day 5).
- Project 1A is **due today**. Before you submit: how do you know it's actually correct?

Today

Passing the test cases you happened to think of is not the same as being **correct**. We'll learn where bugs hide, and how to find them even when they don't crash your program.

What's an “Edge Case”?

An **edge case** is an input at the **boundary** of what your code handles — the places most likely to expose an off-by-one error or an unchecked assumption.

Where bugs cluster

Code is usually correct for a “typical” input in the middle of the range. Bugs hide at the **edges**: empty, one element, exactly full, exactly at a threshold.

A one-line reminder from Day 4

`size_ == capacity_` is the boundary that triggers `reserve()`. Any test that never reaches that exact condition never exercises your growth code at all.

Edge Cases for MiniVector

Scenario	What could go wrong
Empty vector (<code>size_ == 0</code>)	<code>pop_back()</code> / <code>operator[]</code> (0) — UB; <code>at</code> (0) is the only access that throws safely
Push into an empty vector	should <i>not</i> call <code>reserve()</code> — <code>capacity_</code> already starts at 2 (never 0); the first real <code>reserve()</code> comes at the <i>third</i> push
Push exactly at <code>size_ == capacity_</code>	must call <code>reserve()</code> <i>before</i> writing — checking <code>size_ > capacity_</code> instead of <code>==</code> never fires, so <code>data_[size_]</code> writes one past the end
Copy of an empty vector	<code>size_ == 0</code> but <code>capacity_ == 2</code> — copy ctor must still allocate the right-sized array
Self-assignment (<code>a = a;</code>)	the Day 5 bug — deletes <code>data_</code> , then reads through that freed address via <code>other</code>
Chained assignment (<code>a = b = c;</code>)	requires <code>operator=</code> to actually return <code>*this</code>

Skeleton: Write a Boundary Test

Fill in the blanks to test the `size_ == capacity_ boundary` — try this together in class:

```
MiniVector mv(2); // size_ == 0, capacity_ == 2, by construction
mv.push_back(10);
mv.push_back(20);
// mv is now exactly FULL: size_ == capacity_ == 2

mv.push_back(30); // this push MUST trigger reserve()

assert(mv.size() == /* 1. what should size_ be now? */);
assert(mv.capacity() == /* 2. the EXPANDED capacity */);
assert(mv[0] == 10 && mv[1] == 20 && mv[2] == 30); // old data
    survived
```

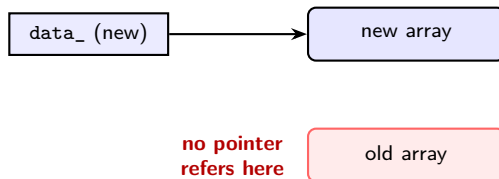
Aside: `MiniVector mv(1);` would work here too — it just moves the first `reserve()` to the second push instead of the third.

Key idea

The last assertion is the one people skip — a buggy `reserve()` can resize correctly and still lose the old elements in the copy.

What Is a Memory Leak?

A **memory leak**: heap memory that is still allocated, but no pointer in your program refers to it anymore — so it can never be freed.



The consequence

That memory is gone for the rest of the program's life. One leak is harmless; a leak inside a loop (like repeated `push_back` calls) can exhaust all available memory.

How MiniVector Could Leak

A buggy `reserve()` that overwrites `data_` **before** freeing the old array:

```
void MiniVector::reserve(std::size_t newCapacity) {  
    int* newData = new int[newCapacity];  
    for (std::size_t i = 0; i < size_; ++i) {  
        newData[i] = data_[i];  
    }  
    data_ = newData;           // BUG: old array's address is now LOST  
    capacity_ = newCapacity; // the delete[] below never runs  
}
```

The moment it's lost

Once `data_ = newData;` runs, nothing in the program still holds the old array's address. It is unreachable, but not freed — a leak, every single time `reserve()` is called.

The fix

`delete[] data_;` **before** reassigning `data_` — the same allocate/copy/free/point order from Day 5.

Leaks Are Silent

The scary part of a memory leak: your program does not crash, and its **output is correct**.

- Every test that checks *values* (“does `mv[2]` equal 30?”) passes — the leaked memory doesn’t affect correctness of what you can observe.
- The only symptom is memory usage that keeps climbing, run after run, call after call.
- You cannot find this by reading console output. You need a tool that watches every `new` and every `delete`.

Recap: What's a Dangling Pointer?

From your lecture-slip questions (Day 5): after `delete[] data_;`, the **memory** is freed, but the **pointer variable** `data_` still holds that old address — now pointing at memory that isn't yours anymore. That's a **dangling pointer**.

Today's new twist

It's not just `data_` itself that can dangle — **anything that points into the array** can too.

Dangling References After reserve()

A reference into MiniVector's array, held across a push_back that triggers growth:

```
MiniVector mv(2);  
mv.push_back(10);  
mv.push_back(20);  
  
int& firstElement = mv[0]; // a reference INTO data_  
  
mv.push_back(30);           // triggers reserve() -- data_ MOVES!  
  
firstElement = 99;          // UNDEFINED BEHAVIOR: writes to freed  
                             memory
```

Why this is so easy to miss

reserve() is completely correct on its own — it frees the *old* array only after copying into the *new* one. The bug is entirely in the caller: holding a reference across a call that might reallocate.

The Fix: Re-fetch After Reallocation

Move the reference to *after* the call that might reallocate:

```
MiniVector mv(2);  
mv.push_back(10);  
mv.push_back(20);  
  
mv.push_back(30);           // triggers reserve() -- data_ MOVES!  
  
int& firstElement = mv[0]; // fetched AFTER the reallocation  
firstElement = 99;         // fine -- points into the live array
```

Why this works

`reserve()` has already finished by the time `firstElement` is created — it resolves against whatever `data_` currently is, not a snapshot from before the reallocation. There's nothing left to dangle.

Your Question: Why Not Just Write `mv[0] = 99;`?

Your question

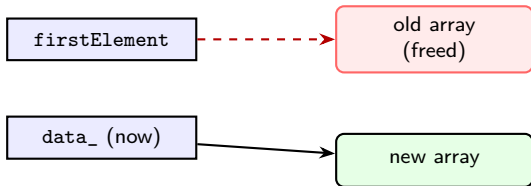
Why hold a reference at all — why not just write `mv[0] = 99;` directly?

For *this one* assignment, that's exactly right — `mv[0] = 99;` works just as well, no reference needed.

The real lesson

As a *user* of `MiniVector`, you can't see `size_` or `capacity_` — they're private, and the growth policy (when it doubles, by how much) is an implementation detail you shouldn't rely on. So you genuinely *can't tell in advance* whether a given `push_back` will reallocate. That's why the rule is unconditional: treat *every* call that could grow the vector as invalidating, every time — re-index fresh instead of holding a reference across it.

Visualizing the Dangling Reference



The rule

Never hold a raw pointer or reference into a `MiniVector`'s elements across any call that might call `reserve()` — re-fetch it afterward instead.

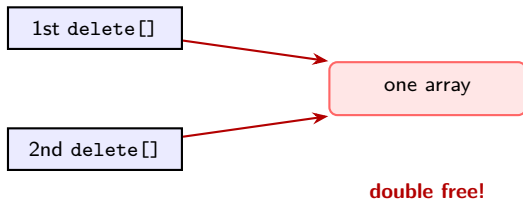
Recap: Double Free From Day 5

You already met one double-free bug: `operator=` without a self-assignment guard deletes `data_`, then reads from that same freed address through `other`.

Today: the direct version

You don't need self-assignment to cause a double free — calling `delete[]` on the **same address** twice, from anywhere, is enough.

Double Delete, Directly



The heap's internal bookkeeping gets corrupted. Unlike a leak, this often **does** crash — but frequently much later, or on a completely unrelated line, making it one of the hardest bugs to trace back to its source.

How Do You Find an Invisible Bug?

Leaks, dangling access, and double frees share one property:

- they often produce **no wrong output** and **no immediate crash**;
- when they *do* crash, it's often far from the actual mistake.

You need a tool that watches memory itself

Not one that checks your program's *answers* — one that checks every new, every delete, and every read or write to heap memory, as it happens.

Valgrind: A Memory Debugger

Valgrind runs your compiled program inside an instrumented virtual machine that tracks every memory access and every allocation/free operation, including:

- whether each block was ever allocated, and by which line of code;
- whether it has since been freed — and whether it's freed more than once;
- every read and write, and whether it lands inside a block you actually own.

The trade-off

Your program runs 10–30× slower under Valgrind — acceptable for testing, not for normal use. In exchange, it catches bugs that might otherwise appear only “sometimes,” on someone else’s machine.

Running Valgrind

Compile with debug symbols, then run your test program through Valgrind instead of directly:

```
g++ -std=c++17 -g MiniVectorTest.cpp MiniVector.cpp -o test
valgrind --leak-check=full ./test
```

- `-g` keeps line-number information, so Valgrind's reports point at your actual source lines.
- `-leak-check=full` lists every leaked block individually, instead of just a total count.

On your own laptop?

For this course, run Valgrind in the Linux environment provided by **GitHub Codespaces**. Native Windows and macOS setups are not supported for this course — see the Codespaces setup guide.

What Is a Fork?

A **fork** is a complete, independent copy of a repository, created inside *your own* GitHub account.

Concretely, for this course

You don't have write access to the instructor's repository, `github.com/tong-yee/cs235_program_f26` — you can view it, but you can't push to it. Forking it creates a brand-new, separate repository at `github.com/<you>/cs235_program_f26`, owned entirely by you.

Your Fork, in Three Facts

- **Not read-only.** It's a normal repository you fully own — full read *and* write access, like any repository you created yourself.
- **Independent.** Changes to the instructor's original repository don't appear in your fork automatically, and changes you make in your fork never touch the original.
- **Permanent, once pushed.** Editing a file only changes your Codespace's local copy — `git add`, `git commit`, `git push` is what saves it into your fork for good.

Your Codespace and Git Workflow

That fork is what your Codespace is built on — here's the same idea as one workflow:

```
Fork course repository → Create Codespace  
    from your fork → edit/test  
    → git add → git commit → git push
```

The mental model

The **Codespace** is a disposable Linux workspace — everything you install and run there, including Valgrind, lives only inside it. Your **fork** is the permanent GitHub repository that survives after the Codespace itself is gone — only what you push is saved there.

Reminder: The Buggy reserve()

The leak report on the next slide comes from this code, shown a few slides back:

```
void MiniVector::reserve(std::size_t newCapacity) {  
    int* newData = new int[newCapacity];  
    for (std::size_t i = 0; i < size_; ++i) {  
        newData[i] = data_[i];  
    }  
    data_ = newData;           // BUG: old array's address is now LOST  
    capacity_ = newCapacity; // the delete[] below never runs  
}
```

Reading a Leak Report

A run of the buggy `reserve()`, after enough `push_back` calls that it has already fired once, then fires again:

```
==12345== HEAP SUMMARY:
==12345==      in use at exit: 16 bytes in 1 blocks
==12345==    total heap usage: 3 allocs, 2 frees, 88 bytes allocated
==12345==
==12345== 16 bytes in 1 blocks are definitely lost in loss record 1
    of 1
==12345==    at 0x...: operator new[](unsigned long)
==12345==    by 0x...: MiniVector::reserve(unsigned long)
==12345==    by 0x...: MiniVector::push_back(int)
```

Reading it

“definitely lost” = a genuine leak — nothing points to this block anymore. 16 bytes is four ints — too big to be the constructor’s starting array (`capacity_` starts at 2, i.e. 8 bytes) — so this block was itself produced by an *earlier* `reserve()` call, then orphaned by a *later* one. Here, `reserve()` really is both the builder and the culprit.

A Second Case: Buffer::grow()

The same shape of bug, in the class you already ran under Valgrind in your Codespace (Buffer.hpp / Buffer.cpp / main.cpp):

```
Buffer::Buffer(std::size_t size)           // <-- allocates HERE
    : data_(new int[size]{}), size_(size) { }

void Buffer::grow(std::size_t newSize) {
    int* newData = new int[newSize]{};
    for (std::size_t i = 0; i < size_; ++i) {
        newData[i] = data_[i];
    }
    data_ = newData; // BUG: old array's address is now LOST
    size_ = newSize;
}
```

Same shape as reserve() — with a twist

Allocate, copy, overwrite data_ without freeing the old array first — same bug as reserve(). Here, grow() runs only *once*, so the lost block can only be the **constructor**'s original array — reserve()'s example showed the opposite case.

Reading Buffer's Leak Report

main.cpp builds a Buffer(4), then calls b.grow(8) — same g++ / valgrind commands as your setup guide:

```
==12345== HEAP SUMMARY:
==12345==      in use at exit: 16 bytes in 1 blocks
==12345== total heap usage: 3 allocs, 2 frees, 73,776 bytes
      allocated
==12345==
==12345== 16 bytes in 1 blocks are definitely lost in loss record 1
      of 1
==12345==    at 0x...: operator new[](unsigned long)
==12345==    by 0x...: Buffer::Buffer(unsigned long)
==12345==    by 0x...: main
```

The trap

The trace points at Buffer::Buffer(...) — *not* at grow(). Did the constructor cause the bug? (total heap usage also counts a large runtime start-up allocation — ignore it; LEAK SUMMARY is what matters.)

Nailing Down the Real Cause

The stack trace under “definitely lost” tells you where a lost block was **born**, not where it **became lost**.

- 1 **Read the size.** 16 bytes = four ints — that matches `Buffer(4)`'s array, not `grow(8)`'s (32 bytes). So the *lost* block is the **original** array.
- 2 **Ask: where else does this pointer get reassigned?**
Search for `data_ =` across the whole class. It happens in exactly one other place: `grow()`.
- 3 **That reassignment, with no `delete[]` first, is the real bug** — even though `grow()` never appears in the stack trace at all.

The general method

Valgrind always names the *allocation* site. *You* trace forward through the code to find where that same pointer gets overwritten — that function is the one responsible for the leak.

Reading an Invalid-Access Report

An off-by-one write past the end of an array:

```
==12345== Invalid write of size 4
==12345==    at 0x...: MiniVector::push_back(int)
==12345== Address 0x51f0050 is 0 bytes after a block of size 8 alloc'
d
==12345==    at 0x...: operator new[](unsigned long)
==12345==    by 0x...: MiniVector::reserve(unsigned long)
```

Reading it

“0 bytes after a block of size 8” — this write landed exactly one int past the end of an 8-byte (two-int) array. Picture a MiniVector already full, `size_ == capacity_ == 2`: a buggy growth check like `size_ > capacity_` (instead of `==`) never fires, so `reserve()` never runs, and `push_back` falls through to `data_[size_] = ...` — writing to `data_[2]`, one slot past the end. Same off-by-one boundary bug as the Edge Cases table. Concretely: the block itself (`data_`) starts at `0x51f0050 - 8 = 0x51f0048`; the invalid write at `0x51f0050` is the very next byte after that block ends.

A Third Case: Two Pointers, One Block

Not every Valgrind error is a leak or an invalid write — a third failure mode, in its simplest possible form:

```
int* arr = new int[10]{};
int* brr = arr;           // brr is a COPY of the pointer -- same address

delete[] arr;
delete[] brr;             // BUG: this block was already freed
```

The shallow-copy problem

`arr` and `brr` hold the *same* address — there's only one heap block, but two independent “owners” that each try to free it. This is exactly the failure mode the Big Three protects a class against.

Tracing the Three Call Sites

```
==8947== Invalid free() / delete / delete[] / realloc()
==8947==    at 0x...: operator delete[](void*)
==8947==    by 0x...: main (double_delete.cpp:7)
==8947== Address 0x4e1e080 is 0 bytes inside a block of size 40 free'
d
==8947==    at 0x...: operator delete[](void*)
==8947==    by 0x...: main (double_delete.cpp:6)
==8947== Block was alloc'd at
==8947==    at 0x...: operator new[](unsigned long)
==8947==    by 0x...: main (double_delete.cpp:4)
```

One error, three answers

Line 7: where the invalid free is happening *now* (`delete[] brr;`).
Line 6: where this exact block was *already* freed (`delete[] arr;`).
Line 4: where it was originally allocated (`new int[10]`). Same size, 40 bytes, every time — confirming `arr` and `brr` really do point at the same block.

Reading a Double-Free Report

Same bug, one level up: a class with **no user-written copy constructor** (so `m2 = m1`; shallow-copies `data_`), both objects then destroyed at the end of `main`:

```
==12345== Invalid free() / delete / delete[] / realloc()
==12345==    at 0x...: operator delete[](void*)
==12345==    by 0x...: MiniVector::~~MiniVector() (MiniVector.cpp:10)
==12345==    by 0x...: main (main.cpp:21)
==12345== Address 0x51f0050 is 0 bytes inside a block of size 16 free
'd
==12345==    at 0x...: operator delete[](void*)
==12345==    by 0x...: MiniVector::~~MiniVector() (MiniVector.cpp:10)
==12345==    by 0x...: main (main.cpp:21)
```

Reading it

Valgrind shows **both** events: where the block was freed the first time, and where the second, invalid free happened — both point at `~MiniVector()`, called twice on the same address because `m1` and `m2` share one `data_`. Exactly the missing-copy-constructor bug from Day 5.

Valgrind Workflow Tips

- Always compile with `-g` first — without it, reports show addresses instead of line numbers.
- Run it on your **edge-case tests**, not just a “happy path” — Valgrind can only report what your test actually exercises.
- Fix the **first** error first. One bad write can corrupt memory that causes several unrelated-looking errors later in the same report.
- A clean run ends with: All heap blocks were freed - no leaks are possible and ERROR SUMMARY: 0 errors.

One More Preview: What About Efficiency?

Your copy constructor and `operator=` are now **correct** — but a deep copy is always $O(n)$: it allocates a new array and copies every element.

A wasted copy

What if the *source* object is a temporary that's about to be destroyed anyway — like a `MiniVector` a function returns by value? Copying its elements just to throw the original away a moment later is wasted work.

Example: A Wasted Copy in Action

Concretely, a function that builds a `MiniVector` locally and returns it *by value*:

```
MiniVector makeRange(int n) {  
    MiniVector result(n);  
    for (int i = 0; i < n; ++i) {  
        result.push_back(i);  
    }  
    return result;    // result is about to be destroyed anyway  
}  
  
MiniVector v = makeRange(1000000);
```

The wasted work

Without move semantics, `result`'s elements get deep-copied into `v` — then `result` itself is destroyed right after. Real compilers often optimize this exact pattern away (return value optimization), but the idea generalizes: whenever a source object won't be needed afterward, move construction lets you skip the copy.

References: & vs. &&

`MiniVector&` and `MiniVector&&` are both **references** — aliases for an existing `MiniVector` — but they bind to different kinds of expressions:

Type	Binds to	Meaning
<code>MiniVector&</code>	a named object	“Borrow this object; it’s still in normal use.”
<code>MiniVector&&</code>	a temporary, or an object explicitly marked movable	“This object may safely be emptied — use it as a resource source.”

Terminology: a named object like this is called an **lvalue**; a temporary is called an **rvalue** — you’ll see these words on the next few slides.

Binding Rules in Action

```
MiniVector a(10); // named object  
  
MiniVector& ref = a; // OK -- a is named  
MiniVector&& rref = MiniVector(10); // OK -- temporary
```

Note: `MiniVector(10)` is the temporary object; `rref` isn't a new object — it's only an *alias* for that same temporary.

This is **not** allowed:

```
MiniVector&& rref = a; // error -- a is a named object
```

Why this is illegal

C++ protects `a`: it's a normal named object you may still want to use later, so it can't bind to `&&` — only a temporary, or an object explicitly wrapped in `std::move`, qualifies.

Move Construction: Transferring, Not Copying

A **move** steals the other object's pointer instead of copying its contents — $O(1)$ instead of $O(n)$:

```
MiniVector::MiniVector(MiniVector&& other) noexcept
: data_(other.data_),           // steal the pointer -- no copy!
  size_(other.size_),
  capacity_(other.capacity_) {
  other.data_ = nullptr;        // leave other in a valid, empty state
  other.size_ = 0;
  other.capacity_ = 0;
}
```

Why setting `other.data_ = nullptr`; matters

`other`'s destructor still runs. If we didn't null it out, *that* destructor would free the array we just stole — a double free, one move constructor away from what you just spent this lecture debugging.

Your Question: Why Also Reset `size_` and `capacity_`?

Your question

“Why set `capacity_ = 0`;;? A positive `capacity_` doesn't work together with a `nullptr` `data_`.”

Exactly right. Suppose `other.capacity_` stayed at 4 while `other.data_` is now `nullptr`:

```
other.push_back(99); // size_(0) < capacity_(4) -- room?
```

The invariant, not just `data_`, matters

`push_back()` only compares `size_` to `capacity_` — it never checks `data_` itself. Stale at 4, `capacity_` makes it think there's room, so it skips `reserve()` and writes into `data_[0]` — `nullptr`, undefined behavior. Resetting `capacity_` to 0 keeps `size_ == capacity_`, so the next push correctly calls `reserve()` first.

Your Question: Doesn't Doubling Break at Zero?

Your question

"If capacity_ and size_ are both 0, reserve() needs to handle this edge case: set capacity_ to 2."

You've found another one. push_back grows by calling reserve(2 * capacity()) — but doubling 0 gives 0:

```
other.push_back(1); // size==capacity==0 -- correctly triggers
                    // reserve()... but 2 * 0 = reserve(0)!
```

Stuck at zero, forever

reserve(0) returns immediately because the requested capacity is not larger than the current capacity. Nothing changes, so every later push_back repeats the same failed growth attempt. The fix mirrors the constructor's own rule: reserve(capacity_ == 0 ? 2 : 2 * capacity_); — doubling only works once there's something to double.

Your Question: When Does the Move Constructor Run?

Your question

“When do we need to call the move constructor?”

You almost never call it by name — it’s selected automatically:

- **From a function return:** `MiniVector v = makeRange(5);` is often constructed directly through copy elision; if elision isn’t used, C++ can move from the returned object.
- **Explicitly**, via `std::move`: `MiniVector v2 = std::move(v1);` tells the compiler to treat `v1` as expendable — the clearest case where a move is guaranteed.

When a move runs, the source is “moved from” — its `data_` is gone; don’t use it again except to reassign or destroy it.

Your Question: Why Does the Move Constructor Need `&&`?

Your question

“Why is `MiniVector&&` in the move constructor’s parameter necessary?”

Because `const MiniVector&` (the copy constructor’s parameter) already binds to *everything* — lvalues and rvalues alike. Without a second, more specific overload, there’d be no way to route rvalues to different code.

What `&&` does

`MiniVector&&` binds *only* to rvalues, so overload resolution picks this constructor over `const MiniVector&` exactly when the argument is expendable. That’s also a promise: it’s safe to gut other here — something the copy constructor could never do to a caller’s real object.

Not on Any Exam Yet

This is a preview, not a requirement

`std::move` and move semantics are **not** required for Project 1, and won't appear on the midterm. We'll return to this properly later in the course (STL Containers, smart pointers) — for now, just recognize the *idea*: sometimes you want to **transfer** ownership of a resource instead of **duplicating** it.

Your Question: More Exercises?

Your question

“Can we get more exercises about this? Thank you!”

Exercise 3 (The Big Three) is exactly this topic, applied to a second class (GradeBook) so you practice the same destructor / copy-constructor / operator= pattern on your own, without MiniVector's code to lean on. More short exercises like it will keep coming as we move through the course — each one is deliberately narrow so you can focus on one idea at a time.

Your Question: Header Comments on Homework?

Your question

“Do we need to add comment with our name and program description in our homework?”

Please do — add a short comment at the top of each file you submit: your name and a one-line description of what the file does. It isn't required for Gradescope to grade your submission (that's tied to your account, not your file's contents), but it's good professional practice, and it helps anyone reading your code later — including you, weeks from now.

FAQ: Will There Be a Practice Midterm or Final?

A question that comes up every semester

Will you post a practice midterm or final?

No practice exam will be posted, for two reasons:

- **Course materials change every semester.** An old exam was written for a different sequence of lectures and projects — it wouldn't reliably reflect what *this* semester covered.
- **A posted practice exam sets the wrong expectation.** If I post one but the real exam doesn't follow it closely, students who studied narrowly from those specific questions feel misled. Study from the actual material — lecture slides, exercises, and projects — that *is* where exam questions come from.

Your Question: Why Two Versions of `[]` and `at()`?

Your question

Why are there two versions of operator `[]` and `at()` in the `MiniVector` class?

Each one is overloaded on `const`:

```
int&      operator[](std::size_t idx);    // non-const mv
const int& operator[](std::size_t idx) const; // const mv

int&      at(std::size_t idx);
const int& at(std::size_t idx) const;
```

Why both are needed

A `const MiniVector&` can only call `const` member functions — without the `const` overload, you couldn't even *read* `mv[i]` from one. The non-`const` version returns a non-`const` reference, so `mv[i] = 5;` works too — but only when `mv` itself isn't `const`.

Your Question: What's the Purpose of a Class Like This?

Your question

“The purpose of `Buffer` is to dynamically allocate an array?”

Close — not a one-time allocation: `Buffer` / `MiniVector` **owns and manages** an array for the object's whole lifetime:

- allocate it (constructor),
- grow it when it's full (`reserve()` / `push_back()`),
- free it automatically at the end of its lifetime (destructor) — this is **RAII**, from Day 5.

Why wrap it in a class at all

Otherwise every piece of code touching the array must remember `new/delete` itself; wrapping it means *one* piece of code gets that right.

Your Question: How Do `size_` and `capacity_` Work Together?

Your question

“How do size and capacity work together?”

- `size_`: elements **logically in use** — what `push_back` has actually added.
- `capacity_`: elements the **current array** can hold before it must be reallocated.
- **Invariant:** `size_ <= capacity_`, always.
- When `size_ == capacity_`, the next `push_back` must call `reserve()` first, or it would write past the end.

Why bother separating them?

If `capacity_` always equaled `size_`, every `push_back` would reallocate. Spare `capacity_` (doubled when you run out) is what makes `push_back` amortized $O(1)$ — see Day 4.

Side by Side: Copy Constructor vs. operator=

Your questions

“Still need more help understanding the copy constructor.”
/ “I’m still a bit confused on copying, the assignment and constructor. The steps are clear, I just didn’t understand the explanation fully.”

	Copy constructor	operator=
Start	owns nothing yet (brand new)	already owns a heap array
Step 1	allocate array, sized to other.capacity_	allocate array, sized to other.capacity_
Step 2	copy other's elements in	copy other's elements in
Step 3	(<i>nothing to free</i>)	free <i>this object's own old</i> array
Step 4	point data_ at new array	point data_ at new array; return *this;
Self-assign?	impossible — object doesn't exist yet	must be safe under self-assignment

They do almost the same work — the copy constructor is simpler only because it never has old memory to clean up first.

Side by Side: `operator=` vs. `reserve()`

`operator=` and `reserve()` look strikingly similar — that's not a coincidence:

	<code>operator=</code>	<code>reserve(n)</code>
Runs because	*this assigned <i>another object's</i> contents	the array is full, <code>push_back</code> needs room
Step 1	allocate array sized to <code>other.capacity_</code>	allocate array sized to <code>n</code>
Step 2	copy <i>other's</i> elements in	copy <i>this object's own</i> elements in
Step 3	free the old array	free the old array
Step 4	point <code>data_</code> at new array; copy <code>size_/capacity_</code>	point <code>data_</code> at new array; <code>capacity_ = n</code>
Data from	a <i>different</i> object	<i>this same object</i> , moving to a bigger array

allocate → copy → free old → point — the same four steps, every time a `MiniVector` replaces its array. Once you've written this correctly once, you've essentially written it for all three places it's needed.

Your Question: What Does the Guard Actually Return?

Your question

“After the self-assignment guard, when `*this` is returned, is `*this` just empty or does it automatically copy `other`? I’m confused because no other code is shown before that return.”

That’s exactly the right thing to be confused about — here’s the resolution: `*this` is never **blank or uninitialized**, and nothing needs to be copied. (That’s the sense of “empty” here — not `nullptr`: `*this` is the object itself, not a pointer.)

Why

When `this == &other`, `*this` and `other` are the *same object in memory* — one object with two names, not two equal objects. It already holds every value it’s supposed to, since nothing ever reset it. The guard just recognizes there’s no work to do and returns the object exactly as it was.

Your Question: A Recap of Today

Your suggestion

“A summary of today’s class would solidify my understanding.”

Good idea — here’s today in one line each; the *Key Takeaways* slide near the end of this deck covers the same ground in more depth:

Test boundaries	typical inputs hide bugs; empty, full, and self-assignment don't
Memory leak	allocated memory nothing points to anymore — silent, correct output
Dangling reference	a pointer/reference that outlives the memory <code>reserve()</code> moved away from
Double free	<code>delete[]</code> on the same address twice — corrupts the heap
Valgrind	watches every allocation, free, and memory access, so you don't have to guess

What's Due

Item	Posted	Due
Project 1A: MiniVector	Tu 9/1	today, Th 9/17
Exercise 3 (Big Three)	Tu 9/15	Tu 9/22

Reminder

Project 1A is due **today**. Run Valgrind on your own test cases in your GitHub Codespace before you submit — a clean report gives strong evidence that *the execution you tested* has no leaks, dangling pointers, or double frees. It can't tell you anything about paths your tests never took.

Key Takeaways

- **Edge cases** — empty, one element, exactly at the capacity boundary, self-assignment — are where bugs hide; test them deliberately.
- A **memory leak**: allocated memory that nothing points to anymore. Silent — correct output, growing memory use.
- A **dangling reference** can outlive the memory it points to any time `reserve()` reallocates — not just through `data_` itself.
- A **double free** corrupts the heap and can crash far from its real cause.
- **Valgrind** watches every allocation and every memory access, catching all three even when your program's output looks correct.
- **Move construction** can transfer ownership in $O(1)$ instead of copying in $O(n)$.

Questions?

See you Tuesday, 9/22 — Abstract Data Types; Singly
Linked List: Nodes & Traversal.

Backup Slides

Complete, uncondensed code and extended Valgrind output for reference.

Backup: Buggy reserve(), Complete

```
// BUGGY -- leaks the old array on every call
void MiniVector::reserve(std::size_t newCapacity) {
    if (newCapacity <= capacity_) {
        return;
    }
    int* newData = new int[newCapacity];
    for (std::size_t i = 0; i < size_; ++i) {
        newData[i] = data_[i];
    }
    data_ = newData;           // old array's address is now unreachable
    capacity_ = newCapacity;
}
```


Backup: Correct reserve(), Complete

```
// CORRECT -- frees the old array before losing the only pointer to
// it
void MiniVector::reserve(std::size_t newCapacity) {
    if (newCapacity <= capacity_) {
        return;
    }
    int* newData = new int[newCapacity];
    for (std::size_t i = 0; i < size_; ++i) {
        newData[i] = data_[i];
    }
    delete[] data_;           // free BEFORE overwriting the pointer
    data_ = newData;
    capacity_ = newCapacity;
}
```

Backup: Full Valgrind Leak Summary

```
==12345== HEAP SUMMARY:
==12345==    in use at exit: 24 bytes in 3 blocks
==12345== total heap usage: 6 allocs, 3 frees, 168 bytes allocated
==12345==
==12345== 24 bytes in 3 blocks are definitely lost in loss record 1
    of 1
==12345==    at 0x4849013: operator new[](unsigned long)
==12345==    by 0x1091AB: MiniVector::reserve(unsigned long)
==12345==    by 0x1090F2: MiniVector::push_back(int)
==12345==    by 0x108F10: main
==12345==
==12345== LEAK SUMMARY:
==12345==    definitely lost: 24 bytes in 3 blocks
==12345==    indirectly lost: 0 bytes in 0 blocks
==12345==    possibly lost: 0 bytes in 0 blocks
==12345==    still reachable: 0 bytes in 0 blocks
==12345==
==12345== ERROR SUMMARY: 1 errors from 1 contexts
```

Backup: A Clean Valgrind Run

What you're aiming for before submitting Project 1:

```
==12345== HEAP SUMMARY:
==12345==    in use at exit: 0 bytes in 0 blocks
==12345== total heap usage: 9 allocs, 9 frees, 312 bytes allocated
==12345==
==12345== All heap blocks were freed -- no leaks are possible
==12345==
==12345== ERROR SUMMARY: 0 errors from 0 contexts
```

What a clean run tells you

in use at exit: 0 bytes and ERROR SUMMARY: 0 errors mean Valgrind found no leaks or invalid memory operations on *this* execution. The matching allocation/free counts support that result.